TUGAS 1

PEMROGRAMAN BERORIENTASI OBJEK

Analisis Sistem Berbasis Objek: Sistem Pengelolaan Jadwal Presentasi Kelompok

BAGIAN 1 - ANALISIS SISTEM

1.1 Sistem Pengelolaan Jadwal Presentasi Kelompok

Analisis sistem ini bertujuan untuk mengidentifikasi komponen utama dalam Sistem Pengelolaan Jadwal Presentasi Kelompok dengan menggunakan pendekatan pemrograman berorientasi objek. Sistem ini dirancang untuk membantu proses pengelolaan kegiatan presentasi secara lebih terstruktur, mulai dari pengaturan peserta, penguji, jadwal pelaksanaan, hingga penilaian hasil presentasi. Fokus utama dalam analisis ini meliputi identifikasi class, atribut, method, serta hubungan antar class yang terdapat dalam sistem.

Class	Atribut	Method
Akun (Inheritance) (SuperClass)	- idAkun - nama	- tampilkanAkun()
Peserta (SubClass)	- nim	- tampilkanPeserta()
Penguji (SubClass)	- kodeDosen	- tampilkanPenguji()
Sesi Presentasi (Class)	- idSesi - topik - waktu - status	- mulaiSesi() - selesaiSesi() - tampilkanSesi()
Penilaian (Class)	- idNilai - idSesi - skor	- tampilkanNilai()
Slot Waktu	-waktu	- tidak ada

1.2 Penerapan Inheritance

Konsep inheritance pada sistem ini diterapkan pada class **Akun** sebagai superclass. Class **Peserta** dan **Penguji** merupakan subclass yang mewarisi atribut dan method dari class Akun. Dengan adanya inheritance, atribut umum seperti identitas pengguna tidak perlu dituliskan ulang pada setiap class, sehingga membuat struktur program menjadi lebih efisien dan terorganisir. Selain mewarisi atribut dari superclass, masing-masing subclass juga memiliki atribut dan method tambahan sesuai dengan perannya dalam sistem.

1.3 Alasan Pemilihan Atribut dan Metode

Pemilihan atribut dan method pada setiap class didasarkan pada kebutuhan sistem agar dapat merepresentasikan data dan perilaku objek secara tepat serta sesuai dengan konsep pemrograman berorientasi objek.

- Pada class **Akun**, atribut seperti *idAkun* dan *nama* digunakan sebagai identitas dasar pengguna dalam sistem. Atribut ini penting untuk membedakan setiap pengguna yang terlibat. Method *tampilkanAkun()* digunakan untuk menampilkan informasi pengguna secara sederhana dan jelas.
- Pada class **Peserta**, atribut *nim* digunakan sebagai identitas unik mahasiswa. Hal ini penting karena setiap peserta harus dapat dibedakan satu sama lain. Method *tampilkanPeserta()* digunakan untuk menampilkan data peserta secara lengkap.
- Pada class **Penguji**, atribut *kodeDosen* digunakan sebagai identitas penguji. Method *tampilkanPenguji()* digunakan untuk menampilkan data penguji.
- Pada class **SesiPresentasi**, atribut *idSesi*, *topik*, *waktu*, dan *status* digunakan untuk mengelola informasi jadwal presentasi. Method *mulaiSesi()* dan *selesaiSesi()* digunakan untuk mengatur status pelaksanaan presentasi, sedangkan *tampilkanSesi()* digunakan untuk menampilkan informasi sesi.
- Pada class **Penilaian**, atribut *idNilai*, *idSesi*, dan *skor* digunakan untuk menyimpan hasil penilaian. Method *tampilkanNilai()* digunakan untuk menampilkan hasil penilaian tersebut.
- Pada class **SlotWaktu**, atribut *idSlot*, *tanggal*, *jamMulai*, dan *jamSelesai* digunakan untuk mengatur waktu pelaksanaan presentasi. Method seperti *tampilkanSlot()* dan *cekKetersediaan()* dapat digunakan untuk menampilkan informasi waktu serta memastikan tidak terjadi bentrok jadwal.

1.4 Hubungan antar Class

Hubungan antar class dalam sistem ini dibagi menjadi beberapa jenis, yaitu inheritance, association, dan aggregation. Setiap jenis hubungan digunakan untuk menggambarkan keterkaitan antar objek dalam sistem agar lebih terstruktur dan mudah dipahami.

a. Inheritance

Konsep *inheritance* digunakan untuk menghindari duplikasi atribut dan method, serta untuk mempermudah pengelolaan struktur class dalam sistem. Pada sistem ini, class **Akun** berperan sebagai class induk (*superclass*) yang memiliki atribut dasar seperti identitas pengguna.

Class **Peserta** dan **Penguji** merupakan turunan (*subclass*) dari class Akun. Hal ini menunjukkan bahwa kedua class tersebut memiliki karakteristik umum sebagai pengguna sistem, sehingga dapat mewarisi atribut dan method yang terdapat pada class Akun. Dengan adanya inheritance, penulisan kode menjadi lebih efisien karena tidak perlu mendefinisikan ulang atribut yang sama pada setiap class turunan.

b. Association

Konsep *association* digunakan untuk menggambarkan hubungan antar class yang saling berinteraksi dalam sistem. Hubungan ini menunjukkan bahwa suatu objek dapat berhubungan atau berkomunikasi dengan objek lainnya dalam menjalankan fungsi tertentu.

Dalam sistem ini, terdapat beberapa hubungan association, antara lain hubungan antara **Peserta** dengan **SesiPresentasi**, yang menunjukkan bahwa peserta mengikuti sesi presentasi yang telah dijadwalkan. Selain itu, terdapat hubungan antara **Penguji** dengan **Penilaian**, dimana penguji bertugas memberikan penilaian terhadap presentasi yang dilakukan. Selanjutnya, terdapat hubungan antara **Penilaian** dengan **SesiPresentasi**, yang menunjukkan bahwa setiap penilaian diberikan untuk sesi presentasi tertentu.

Melalui hubungan association ini, dapat terlihat adanya interaksi antar objek yang mendukung jalannya proses presentasi secara keseluruhan.

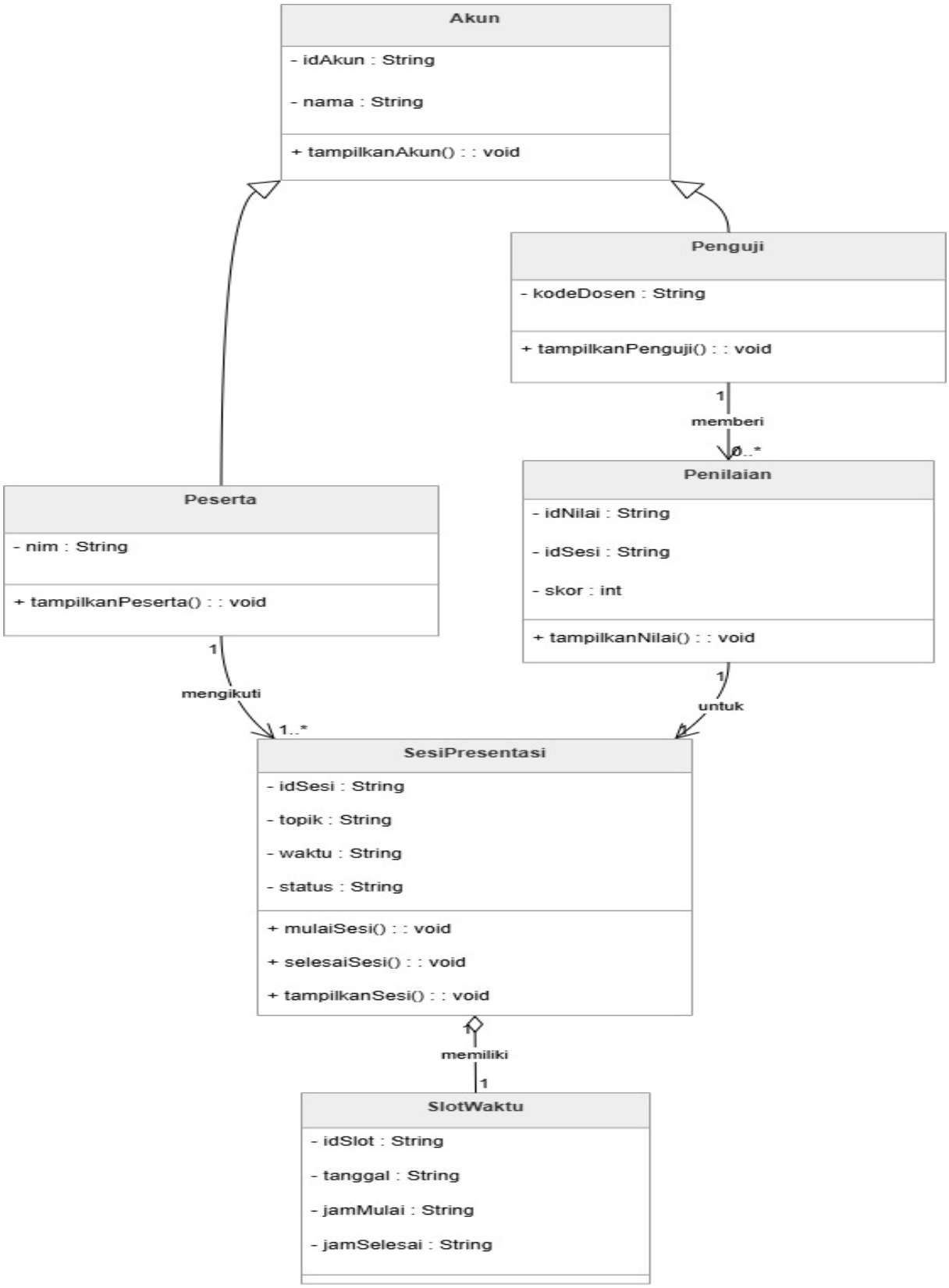
c. Aggregation

Konsep *aggregation* digunakan untuk menggambarkan hubungan kepemilikan antara dua class, namun keduanya tetap dapat berdiri sendiri. Artinya, keberadaan satu objek tidak sepenuhnya bergantung pada objek lainnya.

Pada sistem ini, hubungan aggregation terjadi antara **SesiPresentasi** dengan **SlotWaktu**. Hal ini menunjukkan bahwa setiap sesi presentasi memiliki informasi terkait waktu pelaksanaan yang direpresentasikan oleh slot waktu. Meskipun demikian, slot waktu tetap dapat berdiri sendiri sebagai entitas yang terpisah, sehingga dapat digunakan kembali untuk kebutuhan penjadwalan lainnya.

Penggunaan aggregation ini membantu dalam memisahkan pengelolaan data waktu dengan data sesi presentasi, sehingga sistem menjadi lebih fleksibel dan mudah dikembangkan di masa mendatang.

BAGIAN 2 - DESAIN CLASS DIAGRAM



BAGIAN 3 – Analisis Konsep PBO

3.1 Penerapan Class dan Object dalam Sistem

Konsep *class* pada program ini diterapkan melalui beberapa class seperti Akun, Peserta, Penguji, SesiPresentasi, Penilaian, dan SlotWaktu. Setiap class berfungsi sebagai blueprint yang mendefinisikan atribut dan method sesuai dengan peran masing-masing dalam sistem.
Sedangkan *object* merupakan instansi dari class yang dibuat saat program dijalankan. Contohnya, ketika pengguna menambahkan data peserta atau sesi presentasi, maka program akan membuat object baru dari class yang bersangkutan dan menyimpannya ke dalam ArrayList. Dengan demikian, setiap data yang ditampilkan atau dikelola dalam sistem merupakan object dari class tertentu.

3.2 Penerapan Enkapsulasi

Enkapsulasi dalam program ini diterapkan dengan cara membatasi akses langsung terhadap atribut dan menyediakan method sebagai perantara untuk mengelola data. Beberapa atribut pada class dideklarasikan menggunakan akses modifier seperti *private* atau *protected*, sehingga tidak dapat diakses secara langsung dari luar class. Data hanya dapat diakses melalui method yang telah disediakan, seperti method untuk menampilkan informasi.
Selain itu, perubahan data juga dilakukan melalui method tertentu, sehingga setiap perubahan dapat dikontrol dengan lebih baik. Dengan penerapan enkapsulasi, data dalam sistem menjadi lebih aman dan terstruktur.

3.3 Alasan Pendekatan PBO Lebih Baik dari Prosedural

Pendekatan pemrograman berorientasi objek lebih sesuai digunakan dalam sistem ini karena melibatkan beberapa entitas yang saling berhubungan, seperti peserta, penguji, dan sesi presentasi. Dengan menggunakan PBO, setiap class memiliki tanggung jawab masing-masing sehingga program menjadi lebih terstruktur dan mudah dipahami. Selain itu, konsep inheritance memungkinkan penggunaan kembali kode yang sama tanpa perlu penulisan ulang, sehingga mengurangi duplikasi.
Dibandingkan dengan pendekatan prosedural, PBO lebih fleksibel untuk dikembangkan di masa depan, karena perubahan pada satu bagian tidak akan mempengaruhi keseluruhan sistem secara langsung.

BAGIAN 4 – Refleksi

4.1 Bagian yang paling sulit dalam mengerjakan tugas ini

Bagian yang paling sulit dalam mengerjakan tugas ini adalah menentukan struktur class dan hubungan antar class agar sesuai dengan konsep PBO. Selain itu, memastikan kesesuaian antara diagram, tabel, dan implementasi program juga menjadi tantangan tersendiri.

4.2 Hal baru yang dipelajari tentang konsep PBO

Melalui tugas ini, saya mempelajari bagaimana konsep PBO seperti class, object, inheritance, dan enkapsulasi dapat diterapkan dalam pembuatan program. Saya juga memahami pentingnya membagi sistem ke dalam beberapa class agar lebih terstruktur dan mudah dikelola.

4.3 Fitur yang Akan dikembangkan dari sistem ini

Apabila sistem ini dikembangkan lebih lanjut, terdapat beberapa fitur yang dapat ditambahkan untuk meningkatkan kualitas dan fungsionalitas sistem.
Pertama, pengelolaan kelompok dapat dibuat lebih lengkap, seperti penentuan anggota dan ketua kelompok. Kedua, penjadwalan presentasi dapat dikembangkan menjadi lebih detail dan terstruktur, misalnya dengan pembagian waktu secara otomatis untuk menghindari bentrok jadwal.

Selain itu, sistem juga dapat dilengkapi dengan penyimpanan data berbasis database agar data dapat disimpan secara permanen dan tidak hilang saat program ditutup. Terakhir, tampilan sistem dapat dikembangkan menggunakan antarmuka berbasis GUI agar lebih mudah digunakan dan lebih menarik bagi pengguna.Pengembangan tersebut diharapkan dapat membuat sistem menjadi lebih efisien, terstruktur, dan sesuai dengan kebutuhan penggunaan di dunia nyata.